

▼ Sistemas de numeração

- **Binária:** somente dois dígitos (0..1)
- **Octal:** oito dígitos (0..7)
- **Decimal:** dez dígitos (0..9)
- **Hexadecimal:** dezesseis dígitos (0..F)

Valoração de um número em uma base qualquer na base decimal

Aplica-se a soma dos valores relativos de cada dígito na base que está. Assim, supondo que temos o número binário: **1101**, temos:

- $\text{valor_decimal} = 1 \cdot \text{pow}(2,0) + 0 \cdot \text{pow}(2,1) + 1 \cdot \text{pow}(2,2) + 1 \cdot \text{pow}(2,3)$
- $\text{valor_decimal} = 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 + 1 \cdot 8$
- $\text{valor_decimal} = 1 + 0 + 4 + 8$
- $\text{valor_decimal} = 13$

Suponha agora que o mesmo número (**1101**) acima estivesse na base hexadecimal. Neste caso, a base das potências a ser utilizadas na conversão seria a 16. Ou seja:

- $\text{valor_decimal} = 1 \cdot \text{pow}(16,0) + 0 \cdot \text{pow}(16,1) + 1 \cdot \text{pow}(16,2) + 1 \cdot \text{pow}(16,3)$
- $\text{valor_decimal} = 1 \cdot 1 + 0 \cdot 16 + 1 \cdot 256 + 1 \cdot 4096$
- $\text{valor_decimal} = 1 + 0 + 256 + 4096$
- $\text{valor_decimal} = 4353$

Uso de bases em Python

É possível definir qualquer valor inteiro em Python usando outras bases que não a decimal:

No exemplo abaixo, a primeira variável está especificada em binário (precedida de **0b**). A segunda em decimal. São valores são iguais.

```
v1 = 0b1101 # Especificação em binário v2 = 13
```

Para a base octal precede-se o número de ****0o**** e para a hexadecimal de ****0x**** Os valores da

```
>```\nv1 = 0x2A # Especificação em hexadecimal.\nv2 = 42
```

Na memória tudo é guardado em binário, porém para você programador Python, às vezes é razoável especificar o número na sua forma decimal, octal ou hexadecimal. O Python vai ler e

guardar a forma binária na memória.

Conversão de strings para números (em Python)

Você já deve estar acostumado a ter uma string, como por exemplo, "23" (que lembra um número decimal) e desejar convertê-lo para um inteiro na memória. A operação é simples:

```
a = int("23")
```

Não há dúvida. Na memória está agora guardado o número 23 (ou melhor, está guardado o número 0b10111), já que na memória tudo é guardado em binário e 23 corresponde a 0b10111 (faça a conta e confira!!!)

Mas agora suponha que seu string é este "0x2A", um texto que lembra o número hexadecimal 0x2A. A forma de converter é similar.

```
a = int("0x2A", 16)
```

Na memória está agora guardado o número 0x2A, ou seja 42 em decimal. Melhor ainda, está guardado o número 0b101010, já que na memória tudo é guardado em binário e 0x2A (ou 42) corresponde a 0b101010 (faça a conta e confira!!!)

Conversão de números para strings (em Python)

A forma de converter um número decimal para string é usando *str*.

```
s = str(12)
```

Depois dessa operação o valor de *s* é '12'. Mas há outra maneira, usando formatadores strings.

```
s = f'{12}'
```

A vantagem dessa segunda abordagem, aparentemente mais complexa, é que ela permite especificar que o string resultado deve ser exibido como se o número estivesse em outras bases. Veja os exemplos:

```
s = f'{12:x}' # s terá o 12 na base hexadecimal: 'c'
```

Observe que o string resultante não tem o 0x antes. Assim, se quisermos colocá-lo, o comando será:

```
s = f'0x{12:x}' # s terá o 12 na base hexadecimal: '0xc'
```

Para apresentação na base binária, use *b* no string de formatação. Para octal, use *o*

```
s = f'0o{12:o}' # apresentará o 12 na base octal ('0o14')
```

```
s = f'0b{12:b}' # apresentará o 12 na base binária ('0b1100')
```

Equivalência de bits entre bases

Um número binário é formado por *bits*, cada bit vale 0 ou 1. Com três bits, por exemplo, podemos ter oito possíveis combinações: (000, 001, 010, 011, 100, 101, 110, 111). Os valores desses agrupamentos correspondem, em decimal, a exatamente a 0, 1, 2, 3, 4, 5, 6 e 7. Em outras palavras, um dígito octal ocupa três *bits*.

Dessa forma é fácil converter um número binário para octal por meio do agrupamento de bits (três a três). Exemplo:

- 101011000 (número binário)
- 101 011 000 (bits do número binário agrupados)
- 5 3 0 (dígitos octais equivalentes)
- 530 (número octal equivalente)

Ou seja, em Python, escrever as duas variáveis a seguir têm exatamente o mesmo valor.

```
v1 = 0b101011000 v2 = 0o530
```

Com os números hexadecimais ocorre coisa parecida. Com quatro bits, por exemplo, podemos ter dezesseis possíveis combinações: (0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111). Os valores desses agrupamentos correspondem, em decimal, a exatamente a 0, 1, 2, 3, 4, 5, 6 e 7, 8, 9, 10, 11, 12, 13, 14, 15. Em outras palavras, um dígito hexadecimal ocupa quatro *bits*. Note porém que representamos os números de 10 a 15 por letras A até F.

Portanto, qualquer *byte* pode ser expresso por *exatamente* dois dígitos hexadecimais, já que cada dígito ocupa quatro bits e o byte tem oito bits.

Isso faz com que a base hexadecimal seja muito usada para expressar valores binários, pois em vez de expressar oito dígitos binários, sejam usados apenas dois dígitos hexadecimais.

Por exemplo, endereços IPv6 são formados por oito números, cada um com dezesseis bits. Escrever isso em binário ou mesmo decimal é muito trabalhoso, razão pela qual dígitos hexadecimais sejam usados. Exemplo:

- 101011000 (número binário)
- 1 0101 1000 (bits do número binário agrupados)
- 1 5 8 (dígitos hexadecimais equivalentes)
- 158 (número hexadecimal equivalente)

Números na memória

Seu computador tem 8GB de memória ou 4GB? ou 16GB? Observou que a memória é expressa em *bytes*? Um *byte* tem oito bits. Assim, o maior valor que pode ser guardado em um *byte* é 11111111.

Se você fizer as contas, vai ver que o maior valor em um *byte* é 255. Sendo assim, como o Python (o processador, na verdade) trabalha com números maiores do que 255?

Simples (ou não): agrupando mais de um *byte* para formar um inteiro. Digamos que necessitamos guardar o valor 21579 (em decimal) na memória. Ele pode ser escrito como sendo 101010001001011 em binário. Evidentemente esse número não cabe em um byte (tem mais de oito

bits), mas cabe em dois *bytes*. Assim sendo, na memória teremos dois bytes para esse inteiro. Para evitar exibir o número em binário (que é bastante longo e difícil de ler) diz-se que na memória há o número 0x544b (a representação hexadecimal), ocupando dois bytes na memória.

Tratamento de inteiros em Python

Python, por ser uma linguagem de alto nível, exime o programador de preocupações com o tamanho dos números inteiros na memória. Em verdade, o número de *bytes* ocupados por inteiro do Python pode ser muuuuuito grande.

Porém, às vezes desejamos que um inteiro que caiba em dois *bytes* sejam guardados exatamente em dois *bytes*. Por exemplo, no pacote IP, há um valor para designar o seu tamanho e para isso estão reservados 2 *bytes*, haja vista que o tamanho máximo do pacote IP é 64535 (ou 0xFFFF).

Para contemplar essas situações o Python oferece uma maneira de *forçar* que um inteiro seja colocado em dois *bytes*. Trata-se do uso de *structs*. O seguinte código ilustra esse aspecto.

```
import struct

meuInt = struct.pack("H", 21579)
print (len(meuInt))
```

O valor exibido será 2, pois o "H" no método *pack* basicamente está dizendo para guardar o inteiro 21579 em apenas dois *bytes*.

E quanto ao conteúdo de meuInt? Ali teremos o que denominamos de conjunto de *bytes* (no caso, 2 bytes). Pode-se acessar cada um deles usando operações de indexação normais:

```
print (meuInt[0], meuInt[1])
```

Ao observarmos as partes resultantes desse número pode parecer estranho, pois se obtém 75 e 84 (ambos em decimal). Porém se resolvermos mostrar os números em hexadecimal:

```
print (f'{meuInt[0]:x} {meuInt[1]:x}')
```

Vamos obter: 4b54 (ou seja, exatamente a representação que está na memória - em base hexadecimal). Ocorre que 4b54 corresponde em decimal a exatamente 21579.

Um questão relevante é se é possível exibir todos os bytes de *meuInt* de uma vez só. A resposta é: SIM. Mas é necessário cuidado, pois nessa forma de visualização o Python vai exibir o valor em cada um dos bytes de em duas possíveis maneiras:

- a letra do alfabeto ASCII equivalente ao byte armazenado
- o valor em hexadecimal do byte armazenado, precedido de \x

Assim o comando:

```
print (meuInt)
```

Vai exibir:

```
b'KT'
```

A representação do `b` antes das aspas significa que o Python está exibindo um conjunto de *bytes* e o valor em cada um deles é o código ASCII do *K* e o código ASCII do *T*. Se você consultar a tabela ASCII vai observar que os códigos são respectivamente 4b e 54, como esperado. E porque o Python não exibe o valor hexa direto? Boa pergunta.....

É comum usar um conjunto de bytes para armazenar texto. Nesse caso, cada letra é armazenada com o seu valor número da tabela ASCII (isso é uma simplificação). Na hora de exibir um conjunto de bytes, o Python, na ânsia de ajudar, tenta, sempre que possível decodificar e exibir os *bytes* como letras. Nesse nosso exemplo, fez foi atrapalhar. Em outros casos, vamos constatar que isso ajuda.

Em tendo um conjunto como *meuInt*, que tem dois *bytes*, é possível convertê-lo de volta a um inteiro convencional? Sim, basta desempacotar. Observe:

```
import struct

meuInt = 21579
meuIntBytes = struct.pack("H", meuInt)
meuInt_orig = struct.unpack ("H", meuIntBytes)
print (meuInt_orig)
```

Há muitos detalhes sobre *pack* e *unpack*. Leia a postagem sobre isso. Vai ajudar. Por enquanto vamos ver procedimentos mais simples.

Conversão entre *bytes* e *int*

É possível também converter um conjunto de bytes em um inteiro usando dois métodos *to_bytes* e *from_bytes*. O primeiro desses métodos objetiva converter a representação interna que o Python tem de um *int* em um conjunto de *bytes* que podem representá-los. O segundo dos métodos faz o contrário. Veja um exemplo:

```
meuInt = 21579
meuIntBytes = meuInt.to_bytes(2, 'big')
print (meuIntBytes)
inteiro_original = int.from_bytes (meuIntBytes, 'big')
print (inteiro_original)
```

O primeiro comando apenas coloca o valor 21579 em um inteiro do Python, sem qualquer preocupação de como ele será armazenado. É o que você está acostumado a fazer no uso de programas regulares.

No segundo comando, solicitamos que o Python nos apresente o inteiro *meuInt* na forma de *dois bytes*, em ordem *big-endian*

A seguir temos o processo inverso, possuímos os *bytes* em *big-endian* e queremos que o Python guarde-o na sua representação interna.

Operadores bit-a-bit em Python

No baixo nível é importante aprender as operações binárias *bit-a-bit*, basicamente elas representam a aplicação das operações lógicas bit após bit. Em Python elas são equivalentes às operações lógicas com valores **boolean**, tais como: *and*, *or* e *not*.

A Tabela abaixo mostra a equivalência entre os operadores relacionais e bit-a-bit.

Operador Lógico	Operador bit-a-bit	Lógica
and	&	E
or		OU
not	~	NÃO
^	^	OU EXCLUSIVO

| >> | DESLOCAMENTO DE BITS À DIREITA << | << | DESLOCAMENTO DE BITS À ESQUERDA

A diferença entre os operadores lógicos convencionais e os operadores bit-a-bit é os primeiros são avaliados por curto-circuito, ou seja, a avaliação é feita apenas até a conclusão se o valor representa, sem dúvida, verdadeiro ou falso. As operações bit-a-bit são feitas na sua totalidade.

Operações com *bytes*

Já vimos que o valor 21579 pode ser colocado em dois *bytes* usando *pack*. O resultado é b'KT'.

A* lista* de bytes gerada com *pack* ou explicitamente digitada (b'KT') é *read-only*. Não é possível atualizar os seus valores. Porém é possível criar uma lista de bytes atualizável, criando um *bytearray*. Veja o exemplo.

```
meuInt = b'KT'
meusBytes = bytearray(2) # Cria um bytearray com dois bytes zerados.

# Aplica um xor em cada um dos bytes de meuInt, armazenando em meusBytes
meusBytes[0] = meuInt [0] ^ 2
meusBytes[1] = meuInt [1] ^ 2
print (meusBytes)

# Aplica novamente o xor em cada um dos bytes de meusBytes
meusBytes[0] = meusBytes [0] ^ 2
meusBytes[1] = meusBytes [1] ^ 2

# A aplicação dupla do xor deve voltar aos valores originais
print (meusBytes)
```

Por fim, vejamos um exemplo de cálculo de hashes.

```
import hashlib

m = hashlib.sha256()
m.update(b'JTAG - Jane, Tiago, Amanda, Giovanna')
```

```
hash = m.digest()
print (hash)
```

Faça um programa que conta quantos bits zero há no início do hash.

```
import hashlib

m = hashlib.sha256()
m.update(b'JTAG: Jane, Tiago, Amanda, Giovanna')
hash = m.digest()

nbits = 0
for hb in hash:
    if hb == 0:
        nbits += 8
    else:
        bit = 128
        while (hb & bit) == 0:
            nbits += 1
            bit >>= 1
        break
print (f'sha256: {hash}\nbitsZero iniciais em zero: {nbits}')
```

```
b"\x1dG~\x8d\x009H\x82\xbf\x04\x1d\nd\x1A>=V\x15\\\xcc'\xb4\xb21\xfcE\x05\xba\x14nx"
sha256: b"\x1dG~\x8d\x009H\x82\xbf\x04\x1d\nd\x1A>=V\x15\\\xcc'\xb4\xb21\xfcE\x05\xba\x14
bitsZero iniciais em zero: 3
```

Clique duas vezes (ou pressione "Enter") para editar

Outras Fontes

Isso foi uma introdução à organização e processamento de dados em baixo nível usando Python. É importante consultar a documentação. Leia também:

- <https://www.delftstack.com/pt/howto/python/how-to-convert-bytes-to-integers/>
- <https://www.ti-enxame.com/pt/python/lendo-arquivo-binario-e-loop-sobre-cada-byte/967262432/>

